MODULE 7 · MULTI-TENANCY & SECURITY

Namespaces

A scope boundary for names, quotas, and RBAC — not a network boundary

WHY IT MATTERS

One cluster, carved into many virtual clusters

Teams rarely get a cluster each. A **Namespace** lets many teams, apps, or environments share one cluster without their object names colliding.

It's the scope that quotas, RBAC, and (later) admission policy all key off. Get comfortable with what it scopes — and what it doesn't — before Lab 7.2 builds RBAC on top of it.

BY THE END YOU CAN

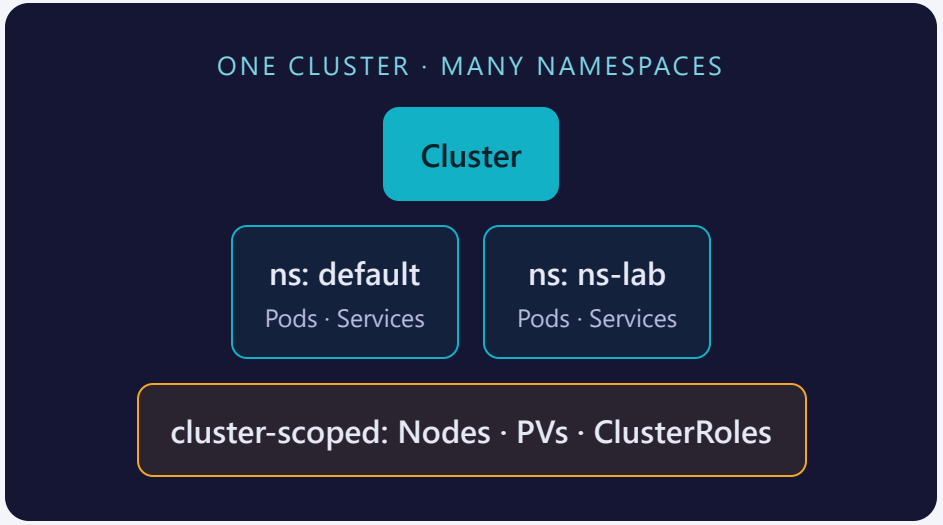
- Explain what a namespace scopes (and can't isolate)
- Name the 4 namespaces that exist by default
- Resolve a Service's DNS name across namespaces
- Switch your `kubectl` default namespace

CONCEPT

A namespace is a scope, not a sandbox

Object **names** only have to be unique within a namespace — two teams can each have a `Deployment` named `web` as long as they're in different namespaces.

A namespace is also the unit that **ResourceQuotas**, **LimitRanges**, and **Roles** attach to. It groups related objects for management — but by itself it draws no line in the network.



WHAT'S NAMESPACED

Most objects live in one — some sit above all of them

NAMESPACED	CLUSTER-SCOPED
Pods, Deployments, ReplicaSets, StatefulSets	Nodes
Services, Endpoints, Ingresses	PersistentVolumes
ConfigMaps, Secrets	Namespaces themselves
Roles, RoleBindings, ResourceQuotas, LimitRanges	ClusterRoles, ClusterRoleBindings, StorageClasses

MENTAL MODEL

If an object needs per-team quota or RBAC, it's namespaced. Infrastructure-level objects — Nodes, storage classes, cluster-wide roles — sit above every namespace.

MANIFEST ANATOMY

The namespace itself is almost nothing — a ResourceQuota gives it teeth

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ns-lab
  labels:
    team: platform
    environment: lab
```

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: ns-quota
  namespace: ns-lab
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

The namespace carries only `metadata.labels` — `metadata.name` is what every `-n` flag and `namespace:` field points at. ResourceQuota is a **separate object** that targets the namespace via `metadata.namespace`; a LimitRange (`labs/7.1/limitrange.yaml`) backfills default requests for Pods that specify none.

DO IT — TWO WAYS

Imperative to explore, declarative to keep

IMPERATIVE — FAST, ONE-OFF	DECLARATIVE — VERSIONABLE, REPEATABLE
`kubectl create namespace ns-lab`	`kubectl apply -f namespace.yaml`
Quick to spin up, no labels attached	Labels, quota, and limit range are all versioned together

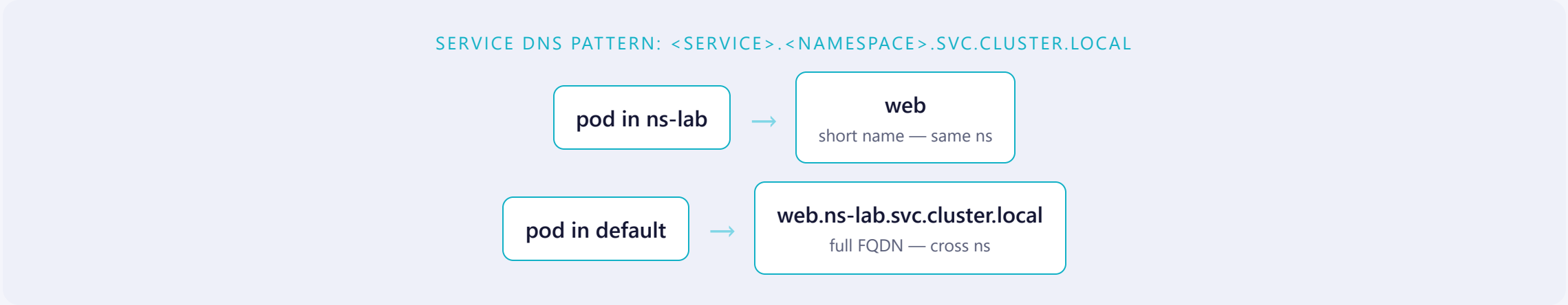
THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl create namespace ns-lab \
    --dry-run=client -o yaml > namespace.yaml
```

SEE IT

Every Service gets a namespace-qualified DNS name



DNS resolves **across** namespaces by default — the boundary isn't a network wall. A pod in `ns-lab` can still resolve (and reach) `kubernetes.default.svc.cluster.local`.

DEFAULT NAMESPACES & CONTEXT

Four namespaces exist before you create any

NAMESPACE	PURPOSE
`default`	Objects created with no `-n` / no `namespace:`
`kube-system`	Control plane & core add-ons (CoreDNS, kube-proxy)
`kube-public`	Readable by everyone, even unauthenticated
`kube-node-lease`	One Lease object per node — fast heartbeats

YOUR CONTEXT HAS A DEFAULT NAMESPACE

```
$ kubectl config set-context --current \
    --namespace=ns-lab
$ kubectl get pods   # no -n needed now
```

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Deleting a namespace deletes everything in it.** Cascading, no confirmation prompt.
- **Not a network boundary.** Pods in different namespaces reach each other by default — see NetworkPolicies (4.7).
- **Forgetting `-n`.** Commands silently land in `default` — or your context's namespace.
- **Cluster-scoped objects reject `namespace:`.** ClusterRole, PersistentVolume, Node must omit it.
- **Quota doesn't fix missing requests.** A LimitRange must be present to backfill defaults for Pods that specify none.
- **Labels on the namespace don't cascade.** Objects inside it don't inherit them automatically.

YOUR NAMESPACE TOOLKIT

Commands to keep at hand

```
$ kubectl get namespaces                                  # list all
$ kubectl create namespace NAME                            # create imperatively
$ kubectl describe namespace NAME                          # quota, limit range, labels
$ kubectl get pods -A                                      # across every namespace
$ kubectl config set-context --current --namespace=NAME # set your default
$ kubectl api-resources --namespaced=true|false            # scope of any kind
```

Tip: `kubectl config view --minify | grep namespace` shows what your current context is pointed at.

RECAP

Namespaces in one breath

- Namespace = scope for names, quotas, RBAC — not a network boundary
- Cluster-scoped objects (Nodes, PVs, ClusterRoles) live outside every namespace
- Services resolve as `<service>.<namespace>.svc.cluster.local`
- `-n` or your context's default namespace decides where commands land

HANDS-ON

Now run `Lab 7.1`

`labs/7.1/`

Next: **7.2 — RBAC**